How the spawn systems and Halo 4 Dominion work

A plain-language walkthrough of the merged update: Invasion spawns, Firefight spawns, the DLC-map script, and Halo 4 Dominion (work in progress).

The shared idea

Every spawn system in this update works the same way. The Python brain decides *where* a dead bot should come back, and sends that spot to the native side with `botctl.spawn_at`. `reach.cpp` then lets the bot spawn on time and teleports its new unit to the spot. Sending NaN calls a pick off, and the game then uses its own spawn. Only bots are ever moved. People are never touched.

Which controller runs depends on the map. `play.py` starts `live_bots.py` when the map has map data, and `escort_bots.py` for Reach Firefight maps without it (the DLC maps) and for ODST. They never run together.

Invasion spawns

Files: `brain/invasion.py`, `tools/live_bots.py` (`res_spawn_check`, `spawn_check`).

Which markers a team uses. The map gives each respawn team a pool per phase. `Inv_res_p1`, `p2` and `p3` markers are usable in that phase only. `Inv_res_zone` hills count from their spawn sequence onward: sequence 1 in phases 1 to 3, sequence 2 in phases 2 and 3, sequence 3 in phase 3 only. When a team has both kinds, each kind is used equally often. The other team's markers are never used.

Which team is which. The map's marker labels name roles, not player colours, and the teams swap sides each round. So `Watch.res_label` reads it off the game itself: at the start of a round, each player team's opening spawns vote for the label whose zone is nearest, and the result is used for the round. If that vote is not clear, the bot falls back to a guess from attackers and defenders. The log marks guessed picks with `[guessed]`. The label is therefore worked out from where the game spawns each team, not read from the marker's own team field.

Picking the spot (`res_zone_pick`).

- **Attackers** favour spawns closer to their objective, so later zones are used more often. A spawn twice as far from the target as the nearest is picked a quarter as often. None is shut out.
- **Defenders** take the free spawn nearest the territory they guard (the core's stand in phase 3).
- **Free vehicles.** A spawn within 25 u of a free, drivable vehicle counts as half its real distance. The bot is then given that vehicle as an errand (30 s) and goes to take it. Going back to an earlier spawn happens only for a vehicle. Vehicles already claimed by another bot's errand or pick are skipped.

Timing. A pick is sent while the bot is dead and redone in a new phase. Picks are held back while a round change is pending, so an old-phase pick cannot hide the new round. At a round change the game's own opening spawns are left alone, because those are what tell the brain the new sides.

Log lines. A layout summary when the map loads, `res spawn: bot ...` for each new pick, a one-time warning when a team has no usable marker, and a note when the Invasion setup is not used.

Firefight spawns on the built-in maps

Files: `tools/live_bots.py`, `brain/firefight_spawns.py`, `native/reach.cpp`.

Teams. Reach Firefight is set up like a custom game: red and blue, up to 7 bots plus you. The Covenant AI count as blue and the human-side AI (the Spartans' allies) as red. A blue bot therefore never aims at, hunts or throws at the Covenant, and red bots and people fight them as enemies.

Recording spawns. The first time each bot is seen alive in a round, its team, position and facing are recorded. Each team builds its own pool of those spots, skipping duplicates within 1 u. Because the game places each team on its own side at the start, each pool is that team's side of the map.

Choosing a respawn. When a bot dies, a spot is picked at random from its own team's pool:

- Spots within 3 u of anyone alive, or on the same floor as them, are dropped.
- Spots 25 u or more from every living enemy (Covenant included) are preferred.
- The pick is sent through `spawn_at`, and `reach.cpp` spawns the bot there.

The pick is random from the team's pool, so a bot does not always return to its own original spot.

What `reach.cpp` does. Any team having lives counts as Firefight, so blue bots respawn as well as red. It spawns the bot at Python's pick and moves its new unit to the same spot. If no pick arrives, it falls back after 10 s: first to where the bot's own team last spawned, then to a teammate's last spawn point, and only then to your spawn, so no bot stays dead.

Round changes. The round watch uses the same signal as Invasion:

1. A transition is pending when 20% or fewer of the lobby is alive.
2. It is confirmed when 60% or more of the lobby (at least 3) respawns within 3 s, or enough births occur inside the 30 s window.
3. Covenant AI are not counted, and the match-start spawn burst is not a round change.
4. While a round is pending, old picks are cancelled with NaN.
5. On confirmation the recorded spots are cleared, and the new round's real bot spawns are recorded.

Checking it. Each recorded bot prints `Firefight: recorded bot N, team T, spawn (x, y, z)`. Red and blue positions should sit on opposite sides of the map. A new round prints `Firefight: round N detected; cleared old spawn pools`.

Firefight on the DLC maps

Files: `tools/escort_bots.py`, `brain/live.py`.

The DLC maps (Unearthed, Installation 04) have no map data, so `escort_bots.py` runs instead. Its bots walk a skilled player's footsteps, stay 3 to 6 u from the person and fight the nearest Covenant.

Team spawns (`TeamSpawns`). Only bots on the team opposing the person are moved:

- Each side's ground is learned from where its AI have stood for at least 8 s: Covenant AI mark team 1, human-side AI mark team 0.
- A pick must lie on ground someone has really walked (`Walked`, which keeps cells of 3 u with the heights seen there).
- It keeps the same clearances as the built-in system: 3 u from anyone, 25 u from foes.
- A pick stands for 25 s and is then dropped. With no AI seen yet, `reach.cpp`'s fallbacks apply.

Hijacking. A bot boards an AI-driven ground vehicle of the other side within 22 u and drives it at the nearest foe. It gives up after 8 s and leaves that vehicle alone for 40 s. `live.ai_riders` is what lets it see AI sitting in vehicles, so a Covenant-driven Ghost, Wraith or Warthog no longer reads as empty.

Firefight bot behaviour

File: `brain/bot.py`.

- **Free roam.** Escort, defender and hunter orders are dropped every tick, so bots roam by their own choice. They pick a random zone centre clear of recently seen enemies (18 u, 32 u when hurt), at least 15 u away when possible, and pick again on arrival, after 30 s, or when an enemy was seen near the spot. Vehicle orders from you still apply.
- **Hold fire.** With no enemy in sight, a bot spends nothing: no rounds, grenades or equipment. Held charges, sprint, melee, reloads, weapon swaps and equipment pressed while taking fire are kept.
- **Ammo.** With no ammo spawns, a bot that is low (its fullest gun under a third of its peak, or under 6 rounds) walks up to 45 u, or 60 u when dry, to take a gun off the ground. This happens only with no enemy in sight.

Halo 4 Dominion (work in progress)

Files: `brain/modes.py`, `brain/dom_base.py`, `brain/bot.py`. This system is still being tuned. The numbers below are the current values.

Bases. Bases are read by label only, so a custom map works like a built-in one. `dom_base` objects are the terminals a player captures by standing at them and holding Use. `dom_base_zone` areas are the stand-in when a map has no terminals. `dom_turret` and `dom_shield` mark what a fortified base spawns.

Who owns a base. Ownership is an estimate. A base becomes a team's once one of its players has stood alone at the terminal for the capture time (2.8 s). A player from the other team there at the same time holds it contested. The game's own owner is not read yet.

Orders (`dominion_orders`). Each tick, each team's bots get a job:

- **Defend:** stand at one of our bases and turn on anyone who comes. At most 35% of a team's bots defend, at most 2 per base, and always fewer than half.

- **Capture:** the rest go to bases that are not ours, spread out with no base taking more than its share.
- If we hold every base, all defend. If they hold every base, all attack.
- A bot keeps its job while it is still valid, so the split does not reshuffle each tick.
- If the other team's turrets or shields stand at a base (within 14 u), they are shot down first. The bot stands off 12 u from the nearest, turrets before shields, and fires only with a clear line within 22 u.

The capture (`dom_base.py`). A base can be held by only one bot at a time, so two bots never fight over a terminal.

1. The bot walks to the terminal and stops within 1.0 u.
2. It settles for 0.4 s with its view on the terminal, tilted slightly down as a person holds it.
3. It holds Use for 5 s. A real capture takes about 2.4 s, so this is a margin. At 1 s into the hold it sets the first hold-level bit, and at 2 s the second, as a real hold does.
4. Every movement, combat and button field is cleared while it holds. Only Use and the view remain.
5. The owner estimate flipping to our team does not end the hold.

Letting go. The claim ends at once when the bot dies, gets into a vehicle, or leaves the 1.0 u radius. After a full hold that did not flip the base, the bot waits 2 s before claiming it again. A claim whose holder has not stepped for 6 s is dropped, so a brief stall does not hand the terminal to another bot. Friendly bases are ignored.

Not stuck, on purpose. While a bot holds a base, the stuck detector is told the stillness is intentional, so it does not wiggle, back off or teach the navigation walls that are not there.

Where to look in the logs

Output	Meaning
`res spawn: bot ...`	An Invasion pick: which marker set, which phase, any free vehicle
`[guessed]`	The respawn label came from the attackers and defenders guess, not the game's opening spawns

Output	Meaning
`Firefight: recorded bot N, team T, spawn (...)`	A bot's first spawn this round, added to its team's pool
`Firefight: round N detected`	A new round was confirmed and the pools cleared
`dominion: capture locked X s`	A bot is mid-hold on a base
`dominion: capture ended after X s: why`	A claim was let go, with the reason